LAB 4: CONVOLUTIONAL NEURAL NETWORKS (CNN)

University of Washington, Seattle

Spring 2023

OUTLINE

Part 1: Introduction to CNN (video link)

- Why do we need CNN?
- Convolutional Layers
- Pooling Layers

Part 2: Applications of CNN (video link)

- Image Segmentation
- Visual Recognition
- CNNs are Leading Algorithms for Visual Recognition
- Image Databases for CNNs

Part 3: CNN Implementation in PyTorch (video link)

- MNIST Classification Example

Lab Assignment (video link)

- LeNet-5 (1988)
- MNIST Classification with LeNet-5

INTRODUCTION TO CNN

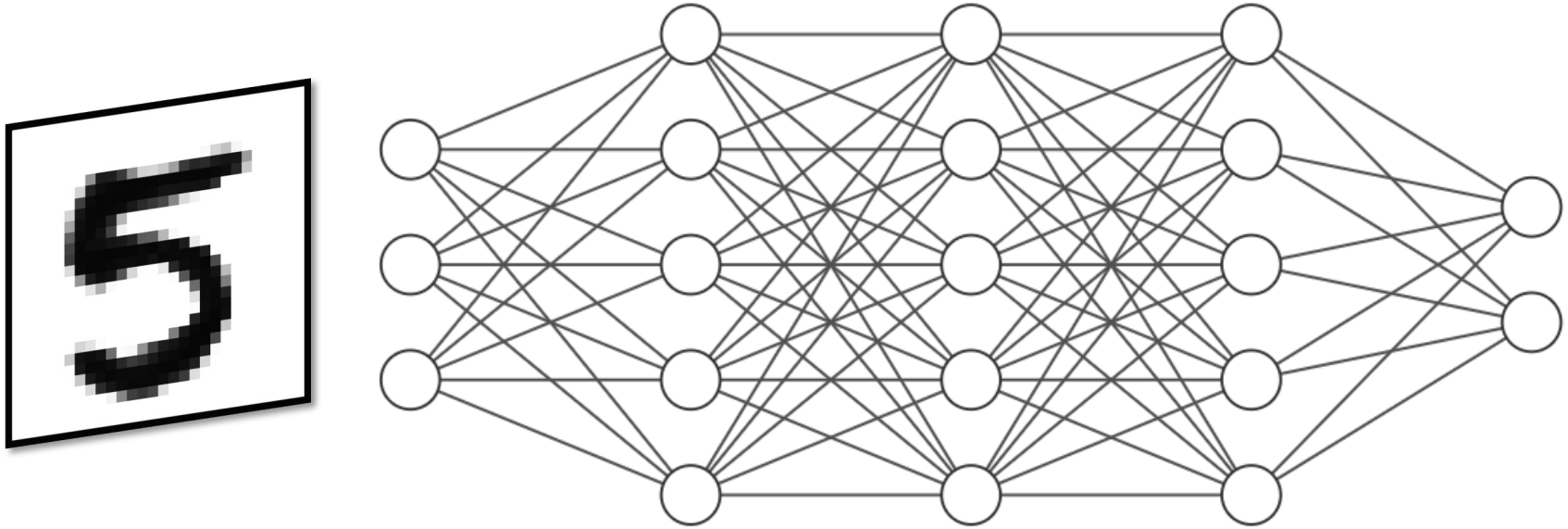
Why do we need CNNs?

Convolutional Layers

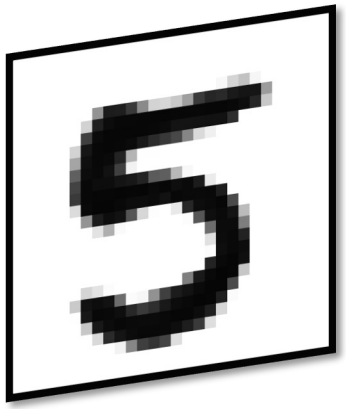
Pooling Layers

Why do we need CNNs?

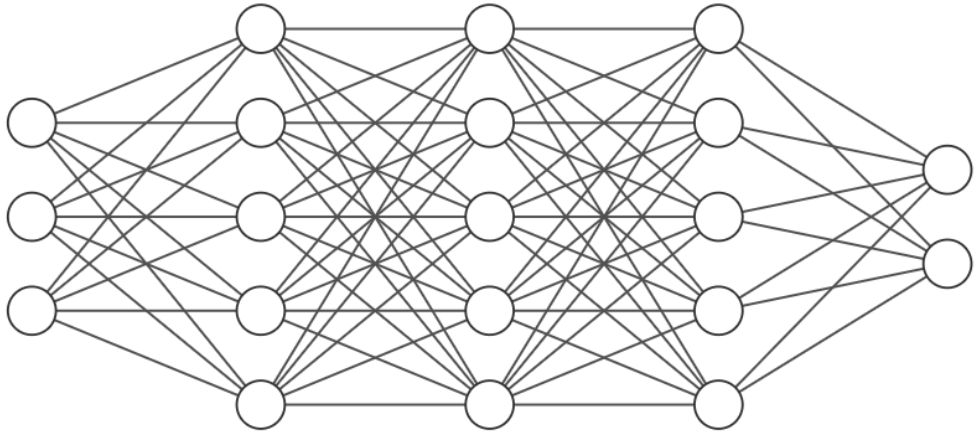
FCN for Image Processing



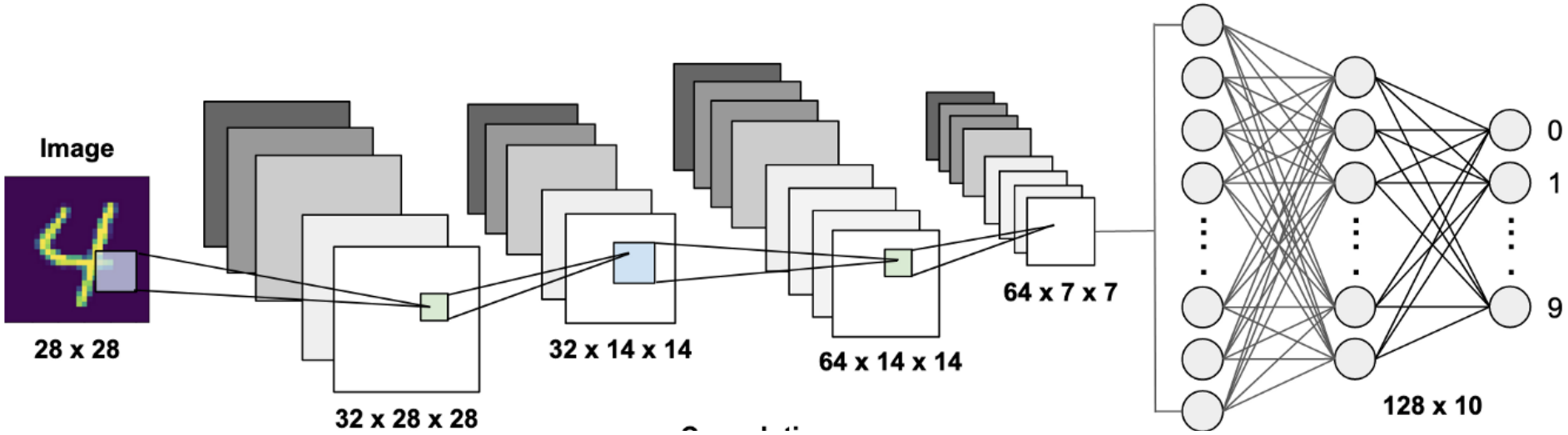
Need for Image Processing Layers



Specialized Layers for Image Processing



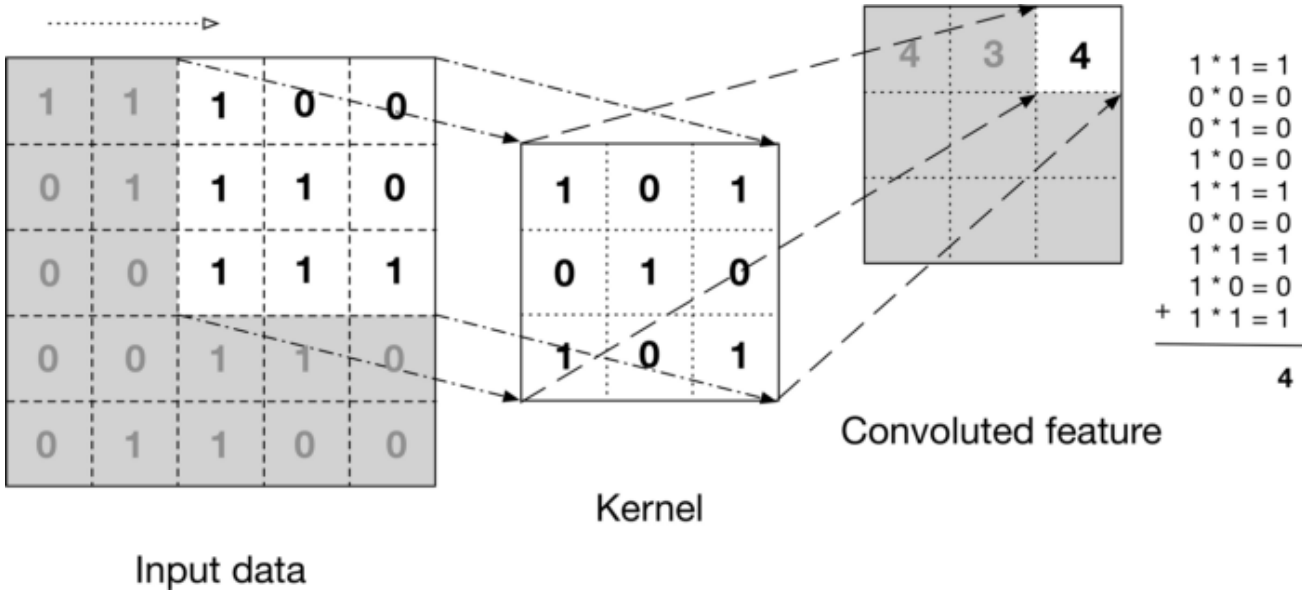
Full CNN Architecture



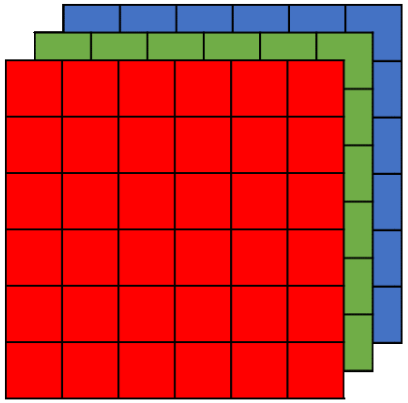
Convolution Layers + Pooling Layers

Convolutional Layers

Image Convolution



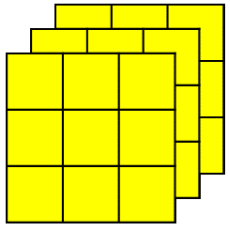
Convolution Kernel



$6 \times 6 \times \boxed{3}$

Input channels

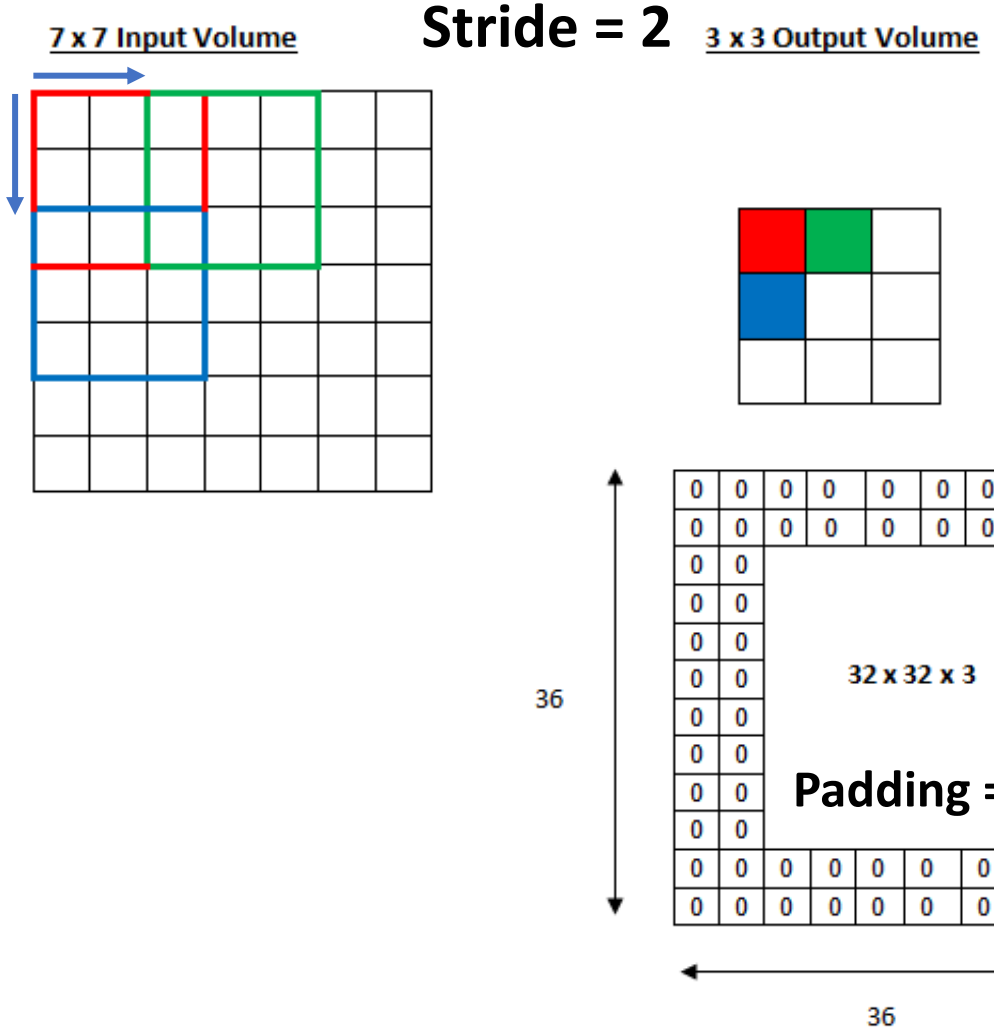
*



$3 \times 3 \times \boxed{3}$

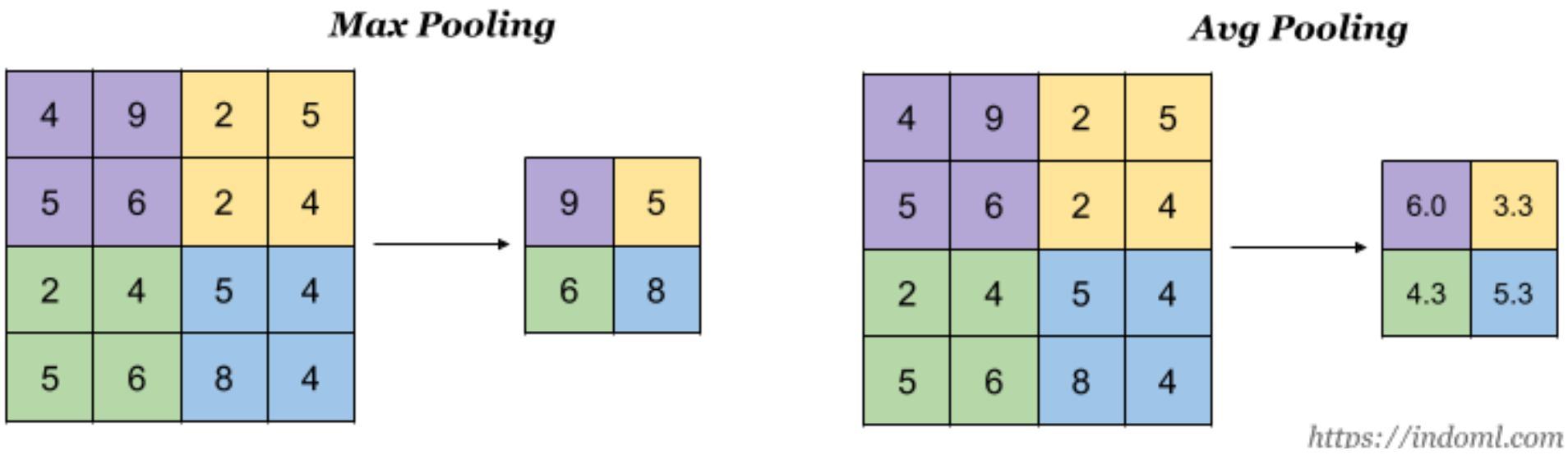
Kernel

Stride and Padding



Pooling Layers

Max Pooling and Average Pooling



APPLICATIONS OF CNNs

Image Segmentation

Visual Recognition

CNNs are Leading Algorithms for Visual Recognition

Image Datasets for CNNs

Image Segmentation

Image Segmentation

- Image analysis procedure of assigning each pixel into a class
- Human brain is capable of high levels of semantic segmentation
- CNNs can be used for semantic segmentation

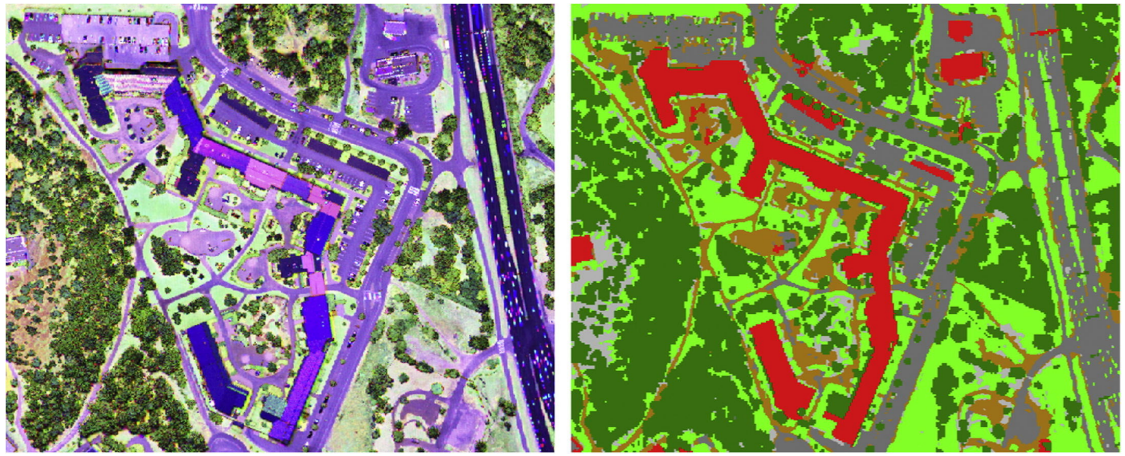
Semantic segmentation

Image credit: https://learnopencv.com/pytorch-for-beginners-semantic-segmentation-using-torchvision/

Image Segmentation Applications

Facial Segmentation

Autonomous Driving



Geo Land Sensing

Image credit: https://learnopencv.com/pytorch-for-beginners-semantic-segmentation-using-torchvision/

Visual Recognition

Visual Recognition Task

- Visual recognition assigns an image into a class

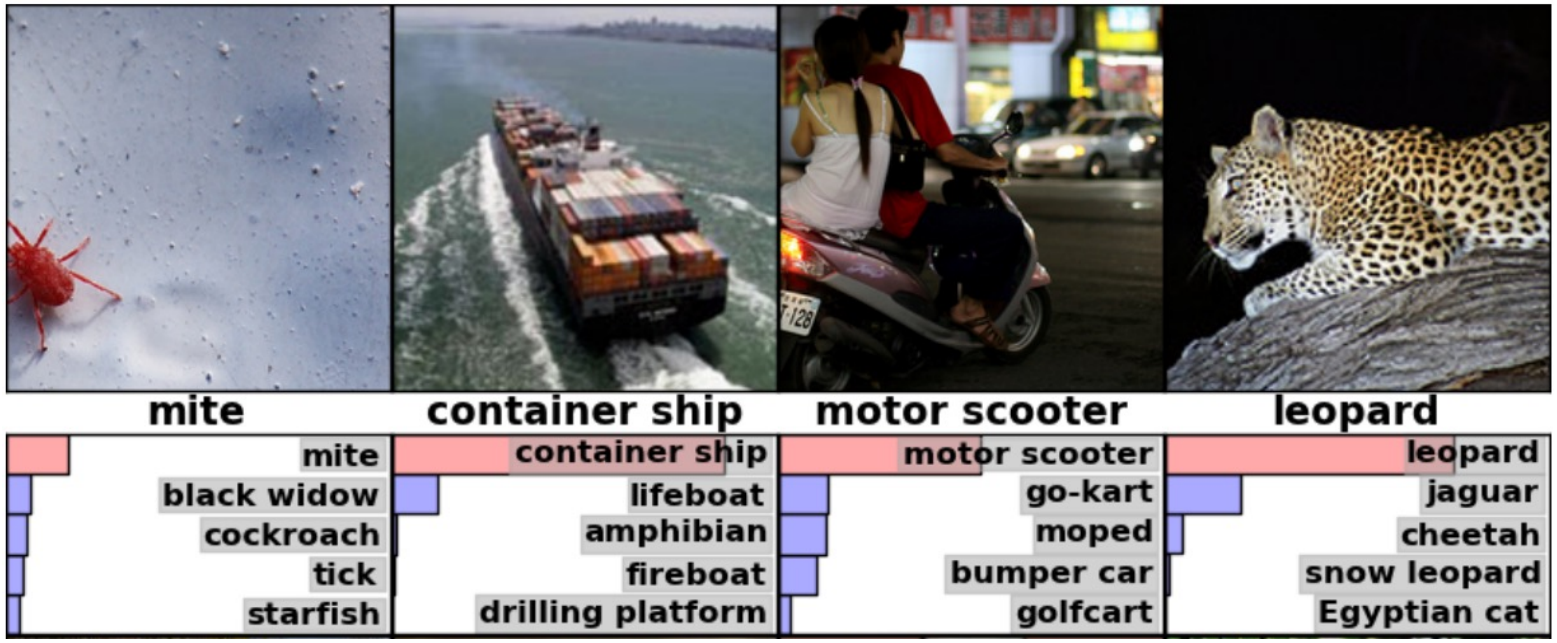
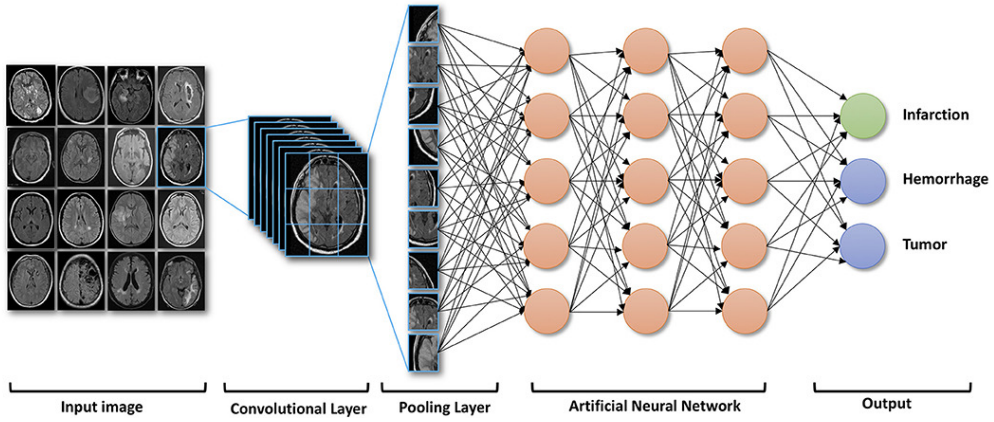
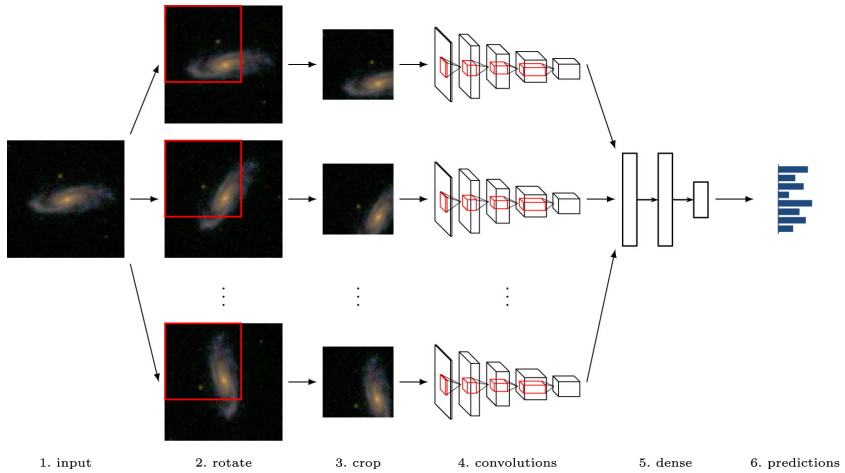


Image credit:
https://papers.nips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

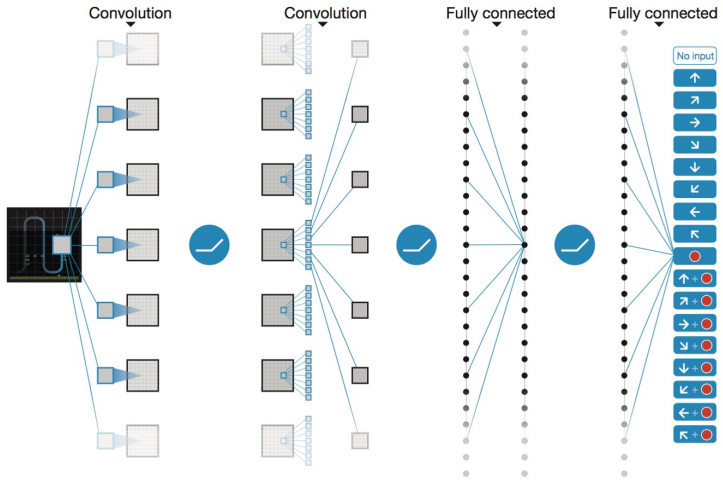
Visual Recognition Applications



Medical Diagnosis



Astronomical Image Analysis



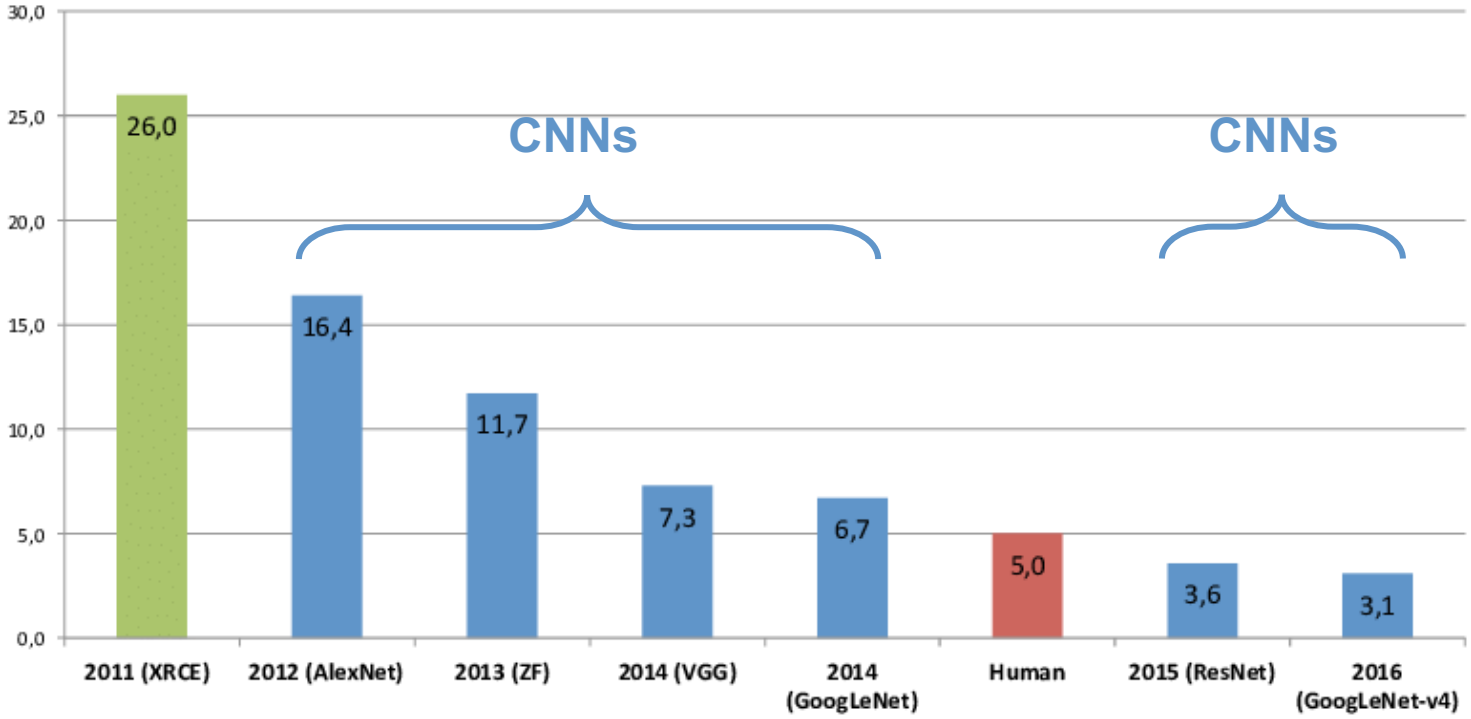
Application in AI Plays Games

Image credits:
https://www.frontiersin.org/articles/10.3389/fneur.2019.00869/full
https://jwuphysics.github.io/blog/galaxies/astrophysics/
https://www.nature.com/articles/nature14236

CNNs are Leading Algorithms for Visual Recognition

Visual Recognition Competition

Image credit: Edge ai + Vision Alliance



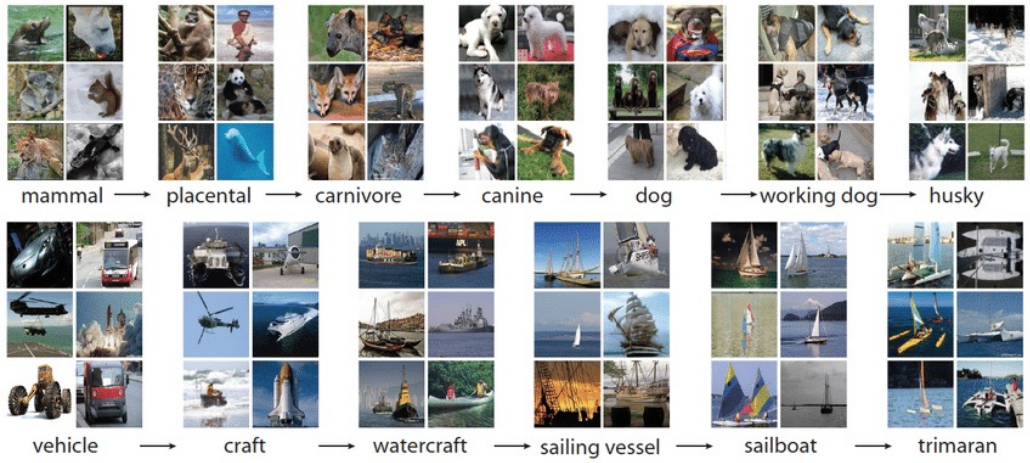
ILSVRC winners over time

Image Databases for CNNs

ImageNet

Large visual database for the visual recognition research

- More than >14 million hand-annotated images
- More than >21k categories
- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) for algorithm evaluation

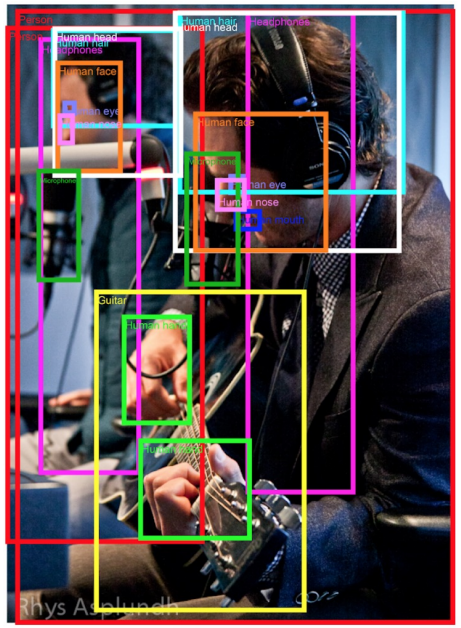


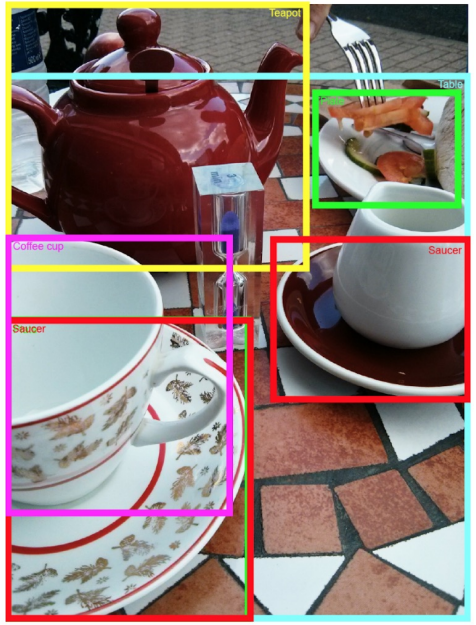
Website: http://www.image-net.org/

Google Open Images

Large-scale object detection, segmentation, and captioning dataset

- >9 mil images annotated
- >16 mil bounding boxes for 600 classes
- Visual relationship annotations (e.g. woman playing guitar)
- Image segmentation for some images





Website: https://storage.googleapis.com/openimages/web/index.html

COCO (Common Objects in Context)

Large-scale object detection, segmentation, and captioning dataset

- 330k images (>200k labeled)
- 1.5 mil object instances
- Object segmentation for all images
- 5 visual relation annotations/image

Website: https://cocodataset.org/#home

CNN IMPLEMENTATION IN PYTORCH

MNIST Classification Example

Prepare Data

```
#Formatting on training set
train_x = X_train.reshape(33600, 1, 28, 28)
train_x  = torch.from_numpy(train_x).float()
# converting the target into torch format
y_train = torch.from_numpy(np.array(y_train))
# shape of training data
train_x.shape, y_train.shape

#Formatting on testing set
X_cv = X_cv.reshape(8400, 1, 28, 28)
X_cv  = torch.from_numpy(np.array(X_cv)).float()
# converting the target into torch format
y_cv = torch.from_numpy(np.array(y_cv))
X_cv.shape, y_cv.shape
```

Define Model

```python
# Create CNN Model
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()

        # Convolution 1
        self.cnn1 = nn.Conv2d(in_channels=1, out_channels=16,
                              kernel_size=3, stride=1, padding=0)
        self.relu1 = nn.ReLU()

        # Max pool 1
        self.maxpool1 = nn.MaxPool2d(kernel_size=2)

        # Convolution 2
        self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32,
                              kernel_size=3, stride=1, padding=0)
        self.relu2 = nn.ReLU()

        # Max pool 2
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)

        # Fully connected 1
        self.fc1 = nn.Linear(32 * 5 * 5, 10)
```

Define Model

```
    def forward(self, x):
        # Input x dimensions:   #nx1x28x28
        # Set 1
        out = self.cnn1(x)      #nx16x26x26
        out = self.relu1(out)
        out = self.maxpool1(out)#nx16x13x13

        # Set 2
        out = self.cnn2(out)    #nx32x11x11
        out = self.relu2(out)
        out = self.maxpool2(out)#nx32x5x5

        #Flatten
        out = out.view(out.size(0), -1) #nx800

        #Dense
        out = self.fc1(out)     #nx10

        return out
```

Select Hyperparameter

```
#Definition of hyperparameters
n_iters = 2500
num_epochs = n_iters / (len(train_x) / batch_size)
num_epochs = int(num_epochs)

# Cross Entropy Loss
error = nn.CrossEntropyLoss()

# SGD Optimizer
model = CNNModel()
learning_rate = 0.001
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

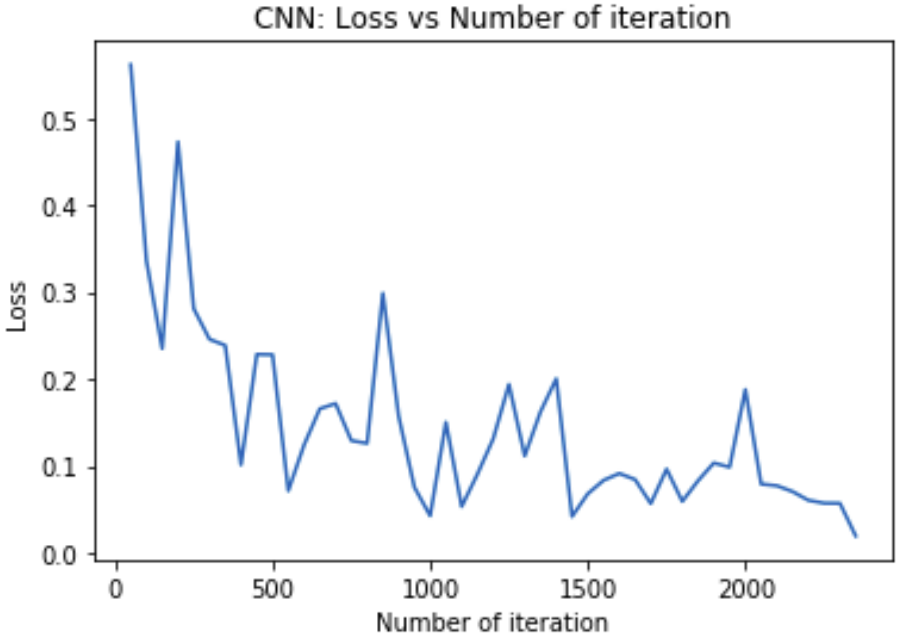
Identify Tracked Values

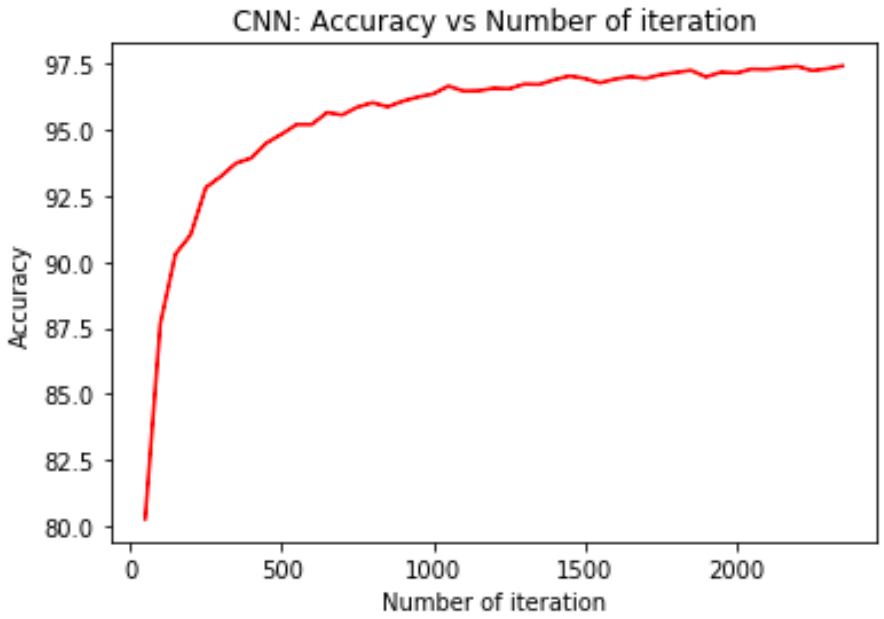
Train Model

```python
# CNN model training
count = 0
loss_list = []
iteration_list = []
accuracy_list = []
for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(train_loader):

        train = Variable(images.view(100,1,28,28))
        labels = Variable(labels)
        # Clear gradients
        optimizer.zero_grad()
        # Forward propagation
        outputs = model(train)
        # Calculate softmax and cross entropy loss
        loss = error(outputs, labels)
        # Calculating gradients
        loss.backward()
        # Update parameters
        optimizer.step()
```

Visualization and Evaluation

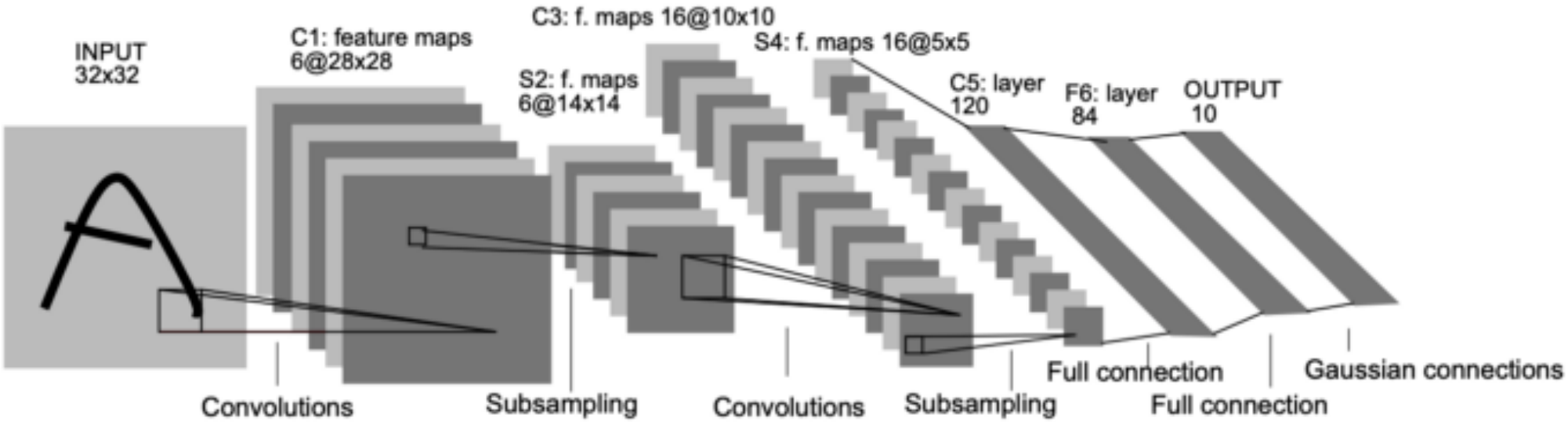




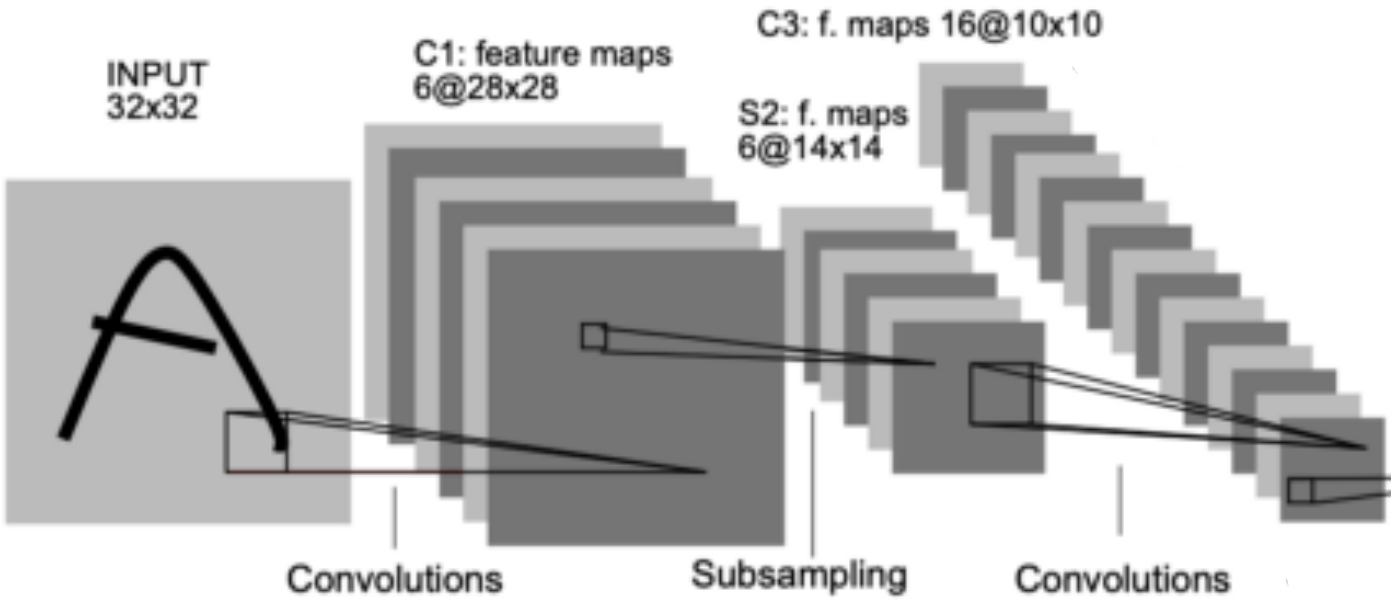
LAB 4 ASSIGNMENT:

MNIST Classification using LeNet-5

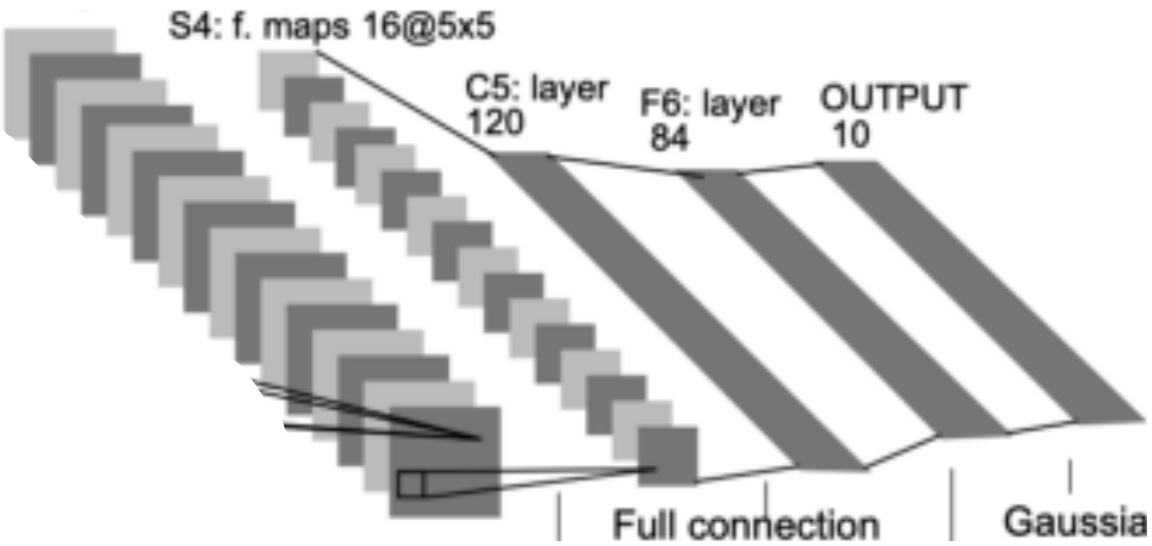
LeNet-5 (1988)


INPUT
32x32
C1: feature maps
6@28x28
S2: f. maps
6@14x14
C3: f. maps 16@10x10
S4: f. maps 16@5x5
C5: layer
120
F6: layer
84
OUTPUT
10
Convolutions
Subsampling
Convolutions
Subsampling
Full connection
Full connection
Gaussian connections

LeNet-5 (1988)


INPUT
32x32
C1: feature maps
6@28x28
S2: f. maps
6@14x14
C3: f. maps 16@10x10
Convolutions
Subsampling
Convolutions

LeNet-5 (1988)


S4: f. maps 16@5x5
C5: layer
120
F6: layer
84
OUTPUT
10
Full connection
Gaussia

Exercise 1: MNIST Classification LeNet-5